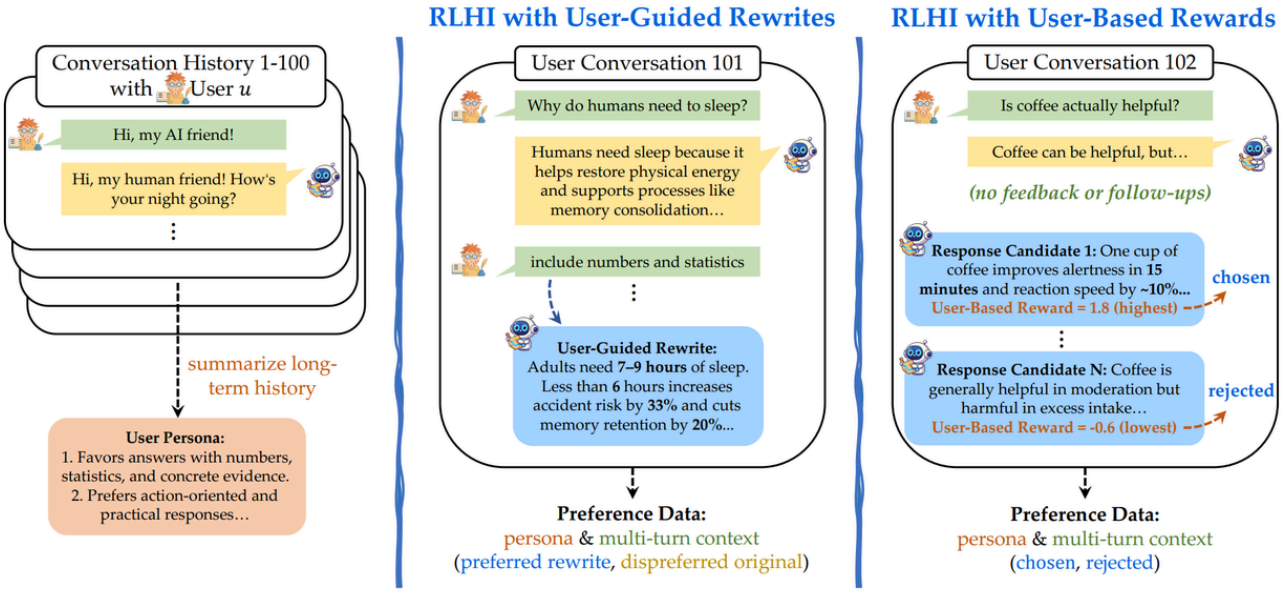
RLHI: Reinforcement Learning from Real-World Human Interaction

> #Finetune #Inference #Preference #DPO



- What problem does this framework solve?
  - With the development of AI, it has already entered **The Era of Real-World Human Interaction**.
  - Not all response generated by the AI is satisfying. Instead, the users frequently let the model regenerate the answer.
    - Based on statistics, 26.51% of the user inputs are "re-attempts with feedback".
  - This provides a good source of RLHF data: more varied and personalized compared to the pre-assembled RLHF datasets
  - This framework provides two ways to exploit human interaction data.
- What does this framework do?
  - It has two parts: RLHI with **User-Guided Rewrites** and RLHI with **User-Based Rewards**.
  - **User-Guided Rewrites**
    - The principle is naive and intuitive: when the user provides a rewrite instruction, this means that the original answer is not satisfying.
    - Regarding the original answer as the negative response, and the newly generated answer as the positive response, we collect multiple such positive-negative pairs and run DPO to finetune the model.
    - It is worth noting that the profile of the user is also provided to the DPO as *a part of system prompt*.

      $$\mathcal{L}_{\text{persona-DPO}} = \mathbb{E}_{u,i}\left[\log\sigma\left(\beta\left(\log\frac{\pi_\theta(y^+_{u,i}|x_{u,i},p_u)}{\pi_{\text{ref}}(y^+_{u,i}|x_{u,i},p_u)} - \log\frac{\pi_\theta(y^-_{u,i}|x_{u,i},p_u)}{\pi_{\text{ref}}(y^-_{u,i}|x_{u,i},p_u)}\right)\right)\right]$$
  - **User-Based Rewards**
    - When there is no clear signal from the user, a user-persona-based reward model can also

be used.

- This reward model knows what the user likes. For example, *"Terry likes quantitative response"*.
- We sample a set of responses, use the reward model to evaluate them one by one, and sort them by their score.
- Afterwards, we select the response with the lowest score as the negative response and the highest score as the positive response to run DPO.

$$\{y_{u,i}^{(n)}\}_{n=1}^{N} \sim \mathcal{M}(x_{u,i}, p_u) \quad \text{then} \quad \begin{cases} y_{u,i}^{+} = \arg\max_{n\in[N]} r(y_{u,i}^{(n)}|x_{u,i}, p_u), \\ y_{u,i}^{-} = \arg\min_{n\in[N]} r(y_{u,i}^{(n)}|x_{u,i}, p_u). \end{cases}$$

- My Thoughts
  - Though this is not a real self-evolving framework, we can make the user feel that the model is optimizing by itself.
  - For **User-Guided Rewrites**, we can have a small model to recognize whether there is a rewrite instruction. If there is one, use a collector function to add the old response and the new response to a buffer.
  - When the # of p-n pairs reaches a threshold t, we can run the DPO in the background to optimize the preference.
- Work Cited
  - Paper
  - Chat History